

# 04. Python Fundamentals (Part-4)

Concepts: Object-Oriented Programming in Python & The 4 Pillars

Author: Golam Maula Lincoln

GitHub Profile: <https://github.com/gmlincoln>

Visit GitHub Profile [↗](https://github.com/gmlincoln)

## 1. Introduction to Object-Oriented Programming (OOP)

Suppose we need to build software to store information related to students studying in our college:

- **Properties:** Name, parent's name, address, graduation year, CGPA, etc.
- **Behaviors:** Fee payment, scholarship calculation, checking minimum attendance criteria, etc.

Without OOP, using basic lists or dictionaries becomes inefficient. With lists, we would have to manage too many parallel structures. With dictionaries, we would re-define keys repeatedly and cannot logically attach behaviors to data.

**Object-Oriented Programming (OOP)** solves this by organizing code into **objects**—bundles of data (attributes) and behavior (methods). It makes software **modular, reusable, and easier to maintain**.

OOP models real-world entities like a *User*, a *Car*, a *Bank Account*, or an *Enemy in a video game*.

**Note:** While OOP is not compulsory for every scenario, it provides a clean, robust structure for complex software architectures.

## 2. Classes & Objects

A **Class** is a blueprint or recipe describing what an object will look like (attributes) and what it can do (methods), but it is not the object itself.

An **Object** (or instance) is a concrete entity created from a class blueprint that occupies memory.

We use the **.** (**Dot Operator**) to access properties and methods of objects.

### Class vs. Object

| Class                                        | Object                                       |
|----------------------------------------------|----------------------------------------------|
| Class is a blueprint / template.             | An object is a concrete instance of a class. |
| Does not exist in memory until instantiated. | Contains actual data & occupies memory.      |
| One class can create any number of objects.  | Each object is independent.                  |

```
class Car:
    brand = "Toyota"

car1 = Car()
car2 = Car()

print(car1.brand) # Output: Toyota
print(car2.brand) # Output: Toyota
```

### 3. Constructors in OOP

In Python, a **constructor** is a special method used to initialize newly created objects. It sets up initial values for object attributes.

We define constructors using the `__init__()` method, which Python calls automatically upon object instantiation.

```
class Student:
    def __init__(self):
        print("constructor was called")

stu1 = Student() # Prints: constructor was called
```

#### The self Parameter

`self` is a special parameter referencing the current **instance of the class** calling the method. Python passes it implicitly during method invocation.

#### Types of Constructors

1. **Default Constructors:** Takes no parameters except `self`.
2. **Parameterized Constructors:** Accepts arguments to uniquely initialize values for each object.

```
class Student:
    def __init__(self, name):
        self.name = name

stu1 = Student("Rahul")
stu2 = Student("Harshita")

print(stu1.name, stu2.name) # Output: Rahul Harshita
```

**Constructor Overloading Note:** Python does not support constructor overloading directly (having multiple `__init__` methods in the same class). Whichever `__init__` method is defined last will overwrite earlier ones.

### 4. Attributes & Methods in OOP

**Attributes** are variables belonging to a class or object that store state. **Methods** are functions defined inside a class representing actions.

#### Types of Attributes

- **Class Attributes:** Belong to the class itself, shared by all objects. Defined outside methods.

```
class Student:
    college = "ABC college" # class attribute

stu1 = Student()
print(stu1.college)
print(Student.college) # Can also be accessed with class name
```

- **Instance Attributes:** Belong individually to each object. Defined inside `__init__()` using `self`.

```
class Student:
    def __init__(self, name, gpa):
        self.name = name # instance attribute
        self.gpa = gpa # instance attribute

stu1 = Student("Rahul", 8.7)
print(stu1.name, stu1.gpa)
```

## Types of Methods

- **Instance Methods:** Take `self` as the first argument. Can access instance and class attributes.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def display(self): # Instance method
        print(f"Name: {self.name}, Marks: {self.marks}")
```

- **Class Methods:** Use `@classmethod` decorator and take `cls` as the first argument. Used to work with class-level data.

```
class Student:
    school_name = "ABC School"

    @classmethod
    def change_school(cls, new_name):
        cls.school_name = new_name
```

- **Static Methods:** Use `@staticmethod` decorator. Do not take `self` or `cls`. Behave like normal functions grouped within the class context.

```
class Math:
    @staticmethod
    def add(a, b):
        return a + b
```

## 5. OOP Pillar 1: Encapsulation

**Encapsulation** is the bundling of data (variables) and methods (functions) into a single unit (class), while controlling access to protect data from accidental or unauthorized modification.

### Access Modifiers

1. **Public Members:** Accessible everywhere, written as normal variables (e.g., `self.name`).
2. **Protected Members:** Indicated by a single underscore `_` (e.g., `self._age`). Suggests "Don't access directly unless needed".

3. **Private Members:** Indicated by a double underscore `__` (e.g., `self.__balance`). Python applies **name mangling** (variable becomes `_ClassName__variable`) to prevent direct external access.

```
class Bank:
    def __init__(self, balance):
        self.__balance = balance # Private variable

b = Bank(5000)
# print(b.__balance) # ERROR: attribute not accessible
print(b._Bank__balance) # Allowed (name-mangled form)
```

## Getters & Setters

Used to read (getters) or update (setters) private members safely.

```
class Employee:
    def __init__(self, salary):
        self.__salary = salary # Private

    def get_salary(self): # Getter
        return self.__salary

    def set_salary(self, new_salary): # Setter
        self.__salary = new_salary

e = Employee(50000)
print(e.get_salary())
e.set_salary(60000)
```

## 6. OOP Pillar 2: Inheritance

**Inheritance** is where one class (child/derived/subclass) acquires properties and behaviors of another class (parent/base/superclass).

Enables code reuse, extensibility, cleaner design, and polymorphism.

### Types of Inheritance

1. **Single Inheritance:** Child inherits from one parent.

```
class Employee: # Parent class
    start_time = "9AM"
    end_time = "5PM"

class Teacher(Employee): # Child class
    def __init__(self, subject):
        self.subject = subject

t1 = Teacher("Data Science")
print(t1.subject, t1.start_time, t1.end_time)
```

2. **Multi-level Inheritance:** Child inherits from a parent, and another class inherits from that child (Grandparent → Parent → Child).

```
class AdminStaff(Employee):
    def __init__(self, role):
        self.role = role

class Accountant(AdminStaff):
    def __init__(self, salary, role):
        super().__init__(role)
        self.salary = salary
```

**3. Multiple Inheritance:** Child inherits from more than one parent class.

```
class Teacher:
    def __init__(self, salary):
        self.salary = salary

class Student:
    def __init__(self, gpa):
        self.gpa = gpa

class TA(Teacher, Student):
    def __init__(self, name, salary, gpa):
        Teacher.__init__(self, salary) # call parent constructor
        Student.__init__(self, gpa)    # call parent constructor
        self.name = name
```

**The super() Keyword:** Used to call parent class methods or constructors from a child class.

## 7. OOP Pillar 3: Abstraction

**Abstraction** is hiding unnecessary implementation details and showing only essential features to the user (e.g., pressing car brakes stops the vehicle without needing hydraulic mechanics knowledge).

### Abstract Classes & Methods

- **Abstract Class:** Cannot be instantiated on its own; acts as a template for child classes.
- **Abstract Method:** Declared but not implemented (child classes **must** override them).

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass

class Lion(Animal):
    def make_sound(self):
        print("Roar!")

class Cow(Animal):
    def make_sound(self):
        print("Moo!")

lion = Lion()
lion.make_sound() # Output: Roar!
```

## 8. OOP Pillar 4: Polymorphism

**Polymorphism** ("poly" = many, "morph" = forms) is the ability of a single function, operator, or object to behave differently based on context.

### Operator Overloading

The same operator performs different operations based on operand data types:

```
print(1 + 2)      # Output: 3 (Adds numbers)
print("1" + "2")  # Output: "12" (Concatenates strings)
```

### Method Overriding

When a child class provides its own version of a method already defined in its parent class (Runtime Polymorphism). Child method takes precedence.

```
class Animal:
    def sound(self):
        print("Some generic sound")

class Dog(Animal):
    def sound(self):
        print("Bark")

a = Animal()
dog = Dog()
a.sound()  # Output: Some generic sound
dog.sound() # Output: Bark
```

### Duck Typing

Idea: "If it looks like a duck and quacks like a duck, it must be a duck."

```
class Dog:
    def speak(self): print("Bark")

class Cat:
    def speak(self): print("Meow")

class Robot:
    def speak(self): print("Beep Boop")

def make_it_speak(entity):
    entity.speak() # doesn't care about explicit type

for e in [Dog(), Cat(), Robot()]:
    make_it_speak(e)
```

✨ Keep Learning & Keep Exploring! ✨

Document created for GitHub profile: [gmlincoln](https://github.com/gmlincoln) (Golam Maula Lincoln)